ENAE464 - 0202

Lab05: Ludwieg-Tube Experiments on a Free-Flying Cone

Due on April 17th, 2026 at 11:59 PM

Dr. Silbaugh, 02:00 PM

Mikołaj Kostrzewa & Vai Srivastava

Experiment Performed: April 10th, 2026
Report Submitted: April 17th, 2026

April 17th, 2026

Enclosed is the technical report for the *Ludwieg-Tube Experiments on a Free-Flying Cone* laboratory experiment. This report presents the experimental methodology, results, and analysis of flow characteristics and effects for a free-flying cone in a Mach-4 Ludwieg tube.
Please feel free to contact us with any questions regarding the contents of this report.

Respectfully,
Mikołaj Kostrzewa & Vai Srivastava

Contents

1 Abstract **1**

2 Introduction **1**
2.1 Theoretical Background . . . 1
2.1.1 Ludwieg Tube Flow Analysis . . . 1
2.1.2 Supersonic Conical Flow Theory . . . 2
2.1.3 Drag Force Determination . . . 3
2.2 Experiment Objectives . . . 3

3 Methodology **3**
3.1 Experimental Apparatus . . . 3
3.2 Operating Procedure . . . 4

4 Results **5**
4.1 Freestream Flow Conditions . . . 5
4.1.1 Step 1: Charge Tube Mach Number . . . 5
4.1.2 Step 2: Post-Expansion Conditions . . . 5
4.1.3 Step 3: Stagnation Conditions . . . 5
4.1.4 Step 4: Freestream Static Conditions . . . 5
4.2 Drag . . . 6
4.2.1 Theoretical . . . 6
4.2.2 Experimental . . . 7
4.3 Shock Angle . . . 7

5 Analysis and Discussion **8**
5.1 Comparison Between Theoretical and Experimental Drag Effects . . . 8
5.2 Viscous Component of Drag . . . 8

6 Summary and Conclusions **9**

7 References **9**

8 Appendices **9**
8.1 Data . . . 9
8.2 Code . . . 9

Figures

3.1.1 Schematic Diagram of a Ludwieg Tube [1]. . . . 4
4.3.1 Theoretical shock shape overlaid on cone edges from experimental imaging. The blue solid lines show the detected cone edges, the green circle marks the calculated vertex position, and the red dashed lines represent the theoretical shock rays at angle $\beta = [TBD]^\circ$ from the freestream direction. . . . 8

Listings

8.2.1 Data Analysis Script. . . . 9

1 Abstract

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

2 Introduction

The Ludwieg tube is a type of impulse wind tunnel capable of generating high-speed supersonic and hypersonic flows efficiently and cost-effectively, albeit with limited test duration [1]. Unlike continuous-flow facilities that require large compressors and extensive power infrastructure, the Ludwieg tube operates on a simple principle: a long charge tube is pressurized to the desired fill conditions, then a fast-acting valve or burst diaphragm is opened, allowing an unsteady expansion wave to propagate upstream while accelerating the gas through a converging-diverging nozzle into the test section. This transient operation provides clean, uniform flow conditions ideal for fundamental aerodynamic studies, though the steady test time is typically limited to tens of milliseconds.

When a body travels through a supersonic or hypersonic flow, the fluid cannot be compressed isentropically ahead of the body due to information traveling only at the local sound speed. Instead, discontinuous shock waves form to rapidly compress the flow and redirect it around the obstacle [2]. For slender axisymmetric bodies such as cones, an attached oblique shock wave emanates from the apex, creating a conical shock surface. A remarkable property of supersonic conical flow is that all flow properties along any ray drawn from the cone vertex remain constant, though they vary from ray to ray as the flow is further compressed between the shock and the body surface [1]. This flow structure is fundamentally different from two-dimensional wedge flows and requires solving the Taylor-Maccoll ordinary differential equation for exact solutions.

Free-flying models offer significant advantages over traditional sting-mounted configurations in wind tunnel testing. Sting supports can introduce flow interference effects that contaminate force measurements, particularly for drag-sensitive geometries or base pressure studies. By eliminating physical support structures, free-flying experiments provide cleaner aerodynamic measurements. However, this approach introduces its own challenges: the model's motion must be tracked with high precision using non-intrusive optical methods, and the short test duration of impulse facilities limits the time available for meaningful acceleration measurements. Recent advances in high-speed digital imaging and automated image processing have made free-flight techniques increasingly practical for quantitative aerodynamic force determination.

2.1 Theoretical Background

2.1.1 Ludwieg Tube Flow Analysis

The flow development in a Ludwieg tube involves a complex interaction between unsteady expansion waves and steady isentropic nozzle flow. When the fast-acting valve opens, an expansion wave propagates upstream into the quiescent charge tube at approximately the local sound speed. This expansion accelerates the gas to a subsonic Mach number M_t determined by the area ratio between the charge tube and nozzle throat according to the area-Mach number relationship:

$$\left(\frac{A_t}{A^*}\right)^2 = \frac{1}{M_t^2}\left[\frac{2}{\gamma+1}\left(1+\frac{\gamma-1}{2}M_t^2\right)\right]^{(\gamma+1)/(\gamma-1)} \tag{2.1.1}$$

where A_t is the charge tube cross-sectional area, A^* is the nozzle throat area, and γ is the specific heat ratio. For this equation, the subsonic solution for M_t must be selected.
The pressure and temperature behind the expansion wave, p_t and T_t, are related to the original fill conditions by:

$$\frac{p_t}{p_{\text{fill}}} = \left(1+\frac{\gamma-1}{2}M_t^2\right)^{-2\gamma/(\gamma-1)} \tag{2.1.2}$$

$$\frac{T_t}{T_{\text{fill}}} = \left(1+\frac{\gamma-1}{2}M_t^2\right)^{-2} \tag{2.1.3}$$

These post-expansion conditions serve as the effective reservoir conditions for the subsequent isentropic expansion through the nozzle. The stagnation pressure and temperature are then:

$$\frac{p_0}{p_t} = \left(1+\frac{\gamma-1}{2}M_t^2\right)^{\gamma/(\gamma-1)} \tag{2.1.4}$$

$$\frac{T_0}{T_t} = 1+\frac{\gamma-1}{2}M_t^2 \tag{2.1.5}$$

Finally, the freestream static conditions in the test section are determined from the calibrated exit Mach number M_∞ using the isentropic relations:

$$\frac{p_0}{p_\infty} = \left(1+\frac{\gamma-1}{2}M_\infty^2\right)^{\gamma/(\gamma-1)} \tag{2.1.6}$$

$$\frac{T_0}{T_\infty} = 1+\frac{\gamma-1}{2}M_\infty^2 \tag{2.1.7}$$

The freestream density and velocity then follow from the ideal gas law and definition of Mach number: $\rho_\infty = p_\infty/(RT_\infty)$ and $V_\infty = M_\infty\sqrt{\gamma RT_\infty}$.

2.1.2 Supersonic Conical Flow Theory

The exact solution for supersonic flow over a cone requires solving the Taylor-Maccoll equation, a nonlinear ordinary differential equation with no closed-form analytical solution. Numerical integration is therefore required to obtain the flowfield properties and surface pressure distribution. However, Rasmussen developed accurate analytical approximations that avoid this computational complexity while maintaining excellent agreement with full numerical solutions for slender cones at high Mach numbers [1].
For the shock angle β (measured from the freestream direction), Rasmussen's approximation is:

$$\frac{\sin(\beta)}{\sin(\theta)_c} \approx \sqrt{\frac{\gamma+1}{2}+\frac{1}{M_\infty^2\sin^2\theta_c}} \tag{2.1.8}$$

where θ_c is the cone half-angle. For the pressure coefficient on the cone surface, the approximation is:

$$\frac{C_p}{\theta_c^2} \approx 1+\frac{(\gamma+1)M_\infty^2}{\theta_c^2+2}+\frac{(\gamma-1)M_\infty^2}{\theta_c^2+2}\ln!\left(\frac{\gamma+1}{2}+\frac{1}{M_\infty^2\theta_c^2}\right) \tag{2.1.9}$$

An important property of slender conical flows is that the pressure coefficient C_p is approximately uniform over the entire cone surface. Consequently, for inviscid flow, C_p is equal to the inviscid drag coefficient $C_{D,\text{inviscid}}$.

2.1.3 Drag Force Determination

For a free-flying body subject only to aerodynamic drag, Newton's second law gives $D = ma_x$, where D is the drag force, m is the body mass, and a_x is the acceleration in the flow direction. If the cone's position is tracked over time using high-speed imaging, a quadratic polynomial can be fitted to the position data:

$$x(t) = a_0 + a_1 t + a_2 t^2 \tag{2.1.10}$$

The constant acceleration is then simply $a_x = 2a_2$. The drag coefficient is defined as:

$$C_D \equiv \frac{2D}{\rho_\infty V_\infty^2 A} \tag{2.1.11}$$

where A is the reference area (cone base area, $A = \pi r^2$). Comparison between the experimentally-determined C_D and the inviscid theoretical prediction from Rasmussen's C_p approximation allows estimation of the viscous drag component, which arises from skin friction and boundary layer displacement effects.

2.2 Experiment Objectives

The specific objectives of this experiment are:

1. Determine the freestream flow conditions (Mach number, pressure, temperature, density, and velocity) in the Ludwieg tube test section from the measured fill conditions using isentropic flow relations and expansion wave theory.
2. Track the position of a free-flying cone over time using high-speed shadowgraph imaging at 10 kHz frame rate and automated edge detection algorithms.
3. Calculate the experimental drag coefficient from the measured cone acceleration by fitting a quadratic polynomial to the position-time trajectory and applying Newton's second law.
4. Compute the theoretical inviscid drag coefficient using Rasmussen's pressure coefficient approximation for supersonic conical flow.
5. Visualize the predicted shock wave geometry and compare the theoretical shock angle with the cone geometry observed in experimental images.
6. Estimate the viscous drag component (skin friction and boundary layer effects) by comparing the experimental drag coefficient with the inviscid theoretical prediction.

3 Methodology

3.1 Experimental Apparatus

The experiment was conducted in the University of Maryland Mach-4 Ludwieg tube, an unheated impulse wind tunnel facility. The apparatus consists of a long charge tube connected to a converging-diverging nozzle through a fast-acting valve, as shown in 3.1.1. The charge tube has a cross-sectional area-to-throat area ratio of $A_t/A^* = 11.4$. The nozzle exit Mach number has been calibrated through previous experiments to be $M_\infty = 4.07$, accounting for the effective area reduction due to boundary layer growth on the nozzle walls.

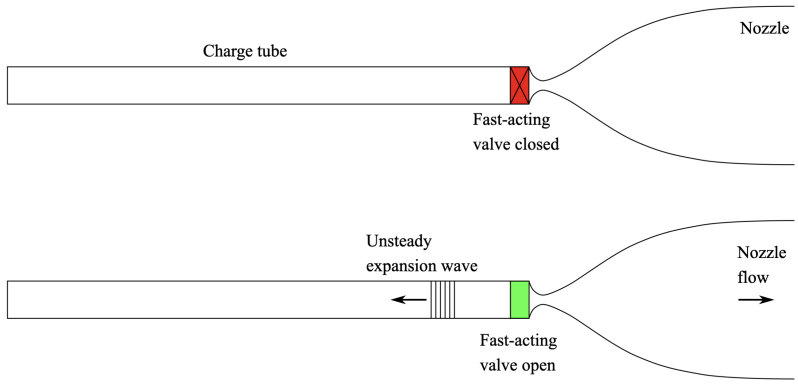

Figure 3.1.1: Schematic Diagram of a Ludwieg Tube [1].

The test article was a slender cone fabricated from lightweight material with a mass of 1.8 g. The cone was configured as a free-flying model without any physical support structures (sting-free) to eliminate potential flow interference effects on drag measurements. The cone half-angle θ_c and base radius r were determined from the edge detection analysis of experimental images, as detailed in 4.2.2. High-speed shadowgraph imaging was employed to visualize the cone and surrounding flowfield, with images captured at a frame rate of 10 kHz (0.1 ms temporal resolution). The imaging system was calibrated to provide a spatial resolution of 12.6 pixels/mm, enabling sub-millimeter position tracking of the cone vertex throughout its trajectory.

3.2 Operating Procedure

The experimental procedure was as follows:

1. The charge tube was filled with air to a pressure of $p_{\text{fill}} = 0.7\,\text{bar}$ (70 kPa) at ambient temperature $T_{\text{fill}} = 296\,\text{K}$.
2. The cone was positioned in the test section on a release mechanism that would deploy it into the freestream upon flow establishment.
3. The high-speed shadowgraph imaging system was configured for 10 kHz frame rate, providing 0.1 ms time resolution between successive images.
4. The fast-acting valve was opened, initiating an unsteady expansion wave that propagated upstream through the charge tube while accelerating the flow through the converging-diverging nozzle.
5. Once steady flow conditions were established in the test section (approximately 5 ms to 10 ms after valve opening), the cone was released and began its free-flight trajectory.
6. A total of 193 frames were captured during the steady flow period, documenting the cone's position and orientation as it accelerated downstream under the influence of aerodynamic drag.
7. The experimental images were post-processed using an automated Python-based edge detection algorithm (ported from the original MATLAB implementation) to extract the upper and lower cone edge coordinates in each frame.
8. Cone vertex positions were determined by fitting linear regressions to the detected edges and computing their intersection point, enabling sub-pixel position accuracy for drag force determination.

4 Results

4.1 Freestream Flow Conditions

The freestream flow conditions in the test section were determined from the measured fill conditions using the equations presented in 2.1. The experimental parameters were:

- Fill pressure: $p_{\text{fill}} = 0.7\,\text{bar} = 70\,000\,\text{Pa}$
- Fill temperature: $T_{\text{fill}} = 296\,\text{K}$
- Charge tube area ratio: $A_t/A^* = 11.4$
- Calibrated exit Mach number: $M_\infty = 4.07$
- Specific heat ratio (air): $\gamma = 1.4$
- Gas constant (air): $R = 287\,\text{J}/(\text{kg}\,\text{K})$

4.1.1 Step 1: Charge Tube Mach Number

Solving 2.1.1 numerically for the subsonic solution yields the Mach number in the charge tube behind the expansion wave:

$$M_t = [TBD \text{ from Python analysis}] \tag{4.1.1}$$

4.1.2 Step 2: Post-Expansion Conditions

Using **??**, the pressure and temperature behind the expansion wave are:

$$p_t = p_{\text{fill}} \left(1 + \frac{\gamma - 1}{2} M_t^2\right)^{-2\gamma/(\gamma-1)} = [TBD]\ \text{Pa} \tag{4.1.2}$$

$$T_t = T_{\text{fill}} \left(1 + \frac{\gamma - 1}{2} M_t^2\right)^{-2} = [TBD]\ \text{K} \tag{4.1.3}$$

4.1.3 Step 3: Stagnation Conditions

Applying **??**, the stagnation conditions are:

$$p_0 = p_t \left(1 + \frac{\gamma - 1}{2} M_t^2\right)^{\gamma/(\gamma-1)} = [TBD]\ \text{Pa} \tag{4.1.4}$$

$$T_0 = T_t \left(1 + \frac{\gamma - 1}{2} M_t^2\right) = [TBD]\ \text{K} \tag{4.1.5}$$

4.1.4 Step 4: Freestream Static Conditions

Finally, using the calibrated exit Mach number $M_\infty = 4.07$ and **??**, the freestream static conditions are:

$$p_\infty = \frac{p_0}{\left(1 + \frac{\gamma-1}{2} M_\infty^2\right)^{\gamma/(\gamma-1)}} = [TBD]\ \text{Pa} \tag{4.1.6}$$

$$T_\infty = \frac{T_0}{1 + \frac{\gamma-1}{2} M_\infty^2} = [TBD]\ \text{K} \tag{4.1.7}$$

$$\rho_\infty = \frac{p_\infty}{RT_\infty} = [TBD] \text{ kg/m}^3 \tag{4.1.8}$$

$$a_\infty = \sqrt{\gamma R T_\infty} = [TBD] \text{ m/s} \tag{4.1.9}$$

$$V_\infty = M_\infty a_\infty = [TBD] \text{ m/s} \tag{4.1.10}$$

These calculated freestream conditions provide the reference state for all subsequent drag coefficient and Reynolds number calculations. The complete results are summarized in 4.1.1.

Table 4.1.1: Calculated freestream flow conditions. Values in red will be filled from Python analysis output.

Parameter	**Value**	**Units**
M_t	[TBD]	—
p_t	[TBD]	Pa
T_t	[TBD]	K
p_0	[TBD]	Pa
T_0	[TBD]	K
M_∞	4.07	—
p_∞	[TBD]	Pa
T_∞	[TBD]	K
ρ_∞	[TBD]	kg/m^3
a_∞	[TBD]	m/s
V_∞	[TBD]	m/s

4.2 Drag

4.2.1 Theoretical

The theoretical inviscid drag coefficient was calculated using Rasmussen's pressure coefficient approximation (2.1.9). The required cone geometry parameters—half-angle θ_c and base radius r—were extracted from the edge detection analysis of the experimental images, as described in 4.2.2.

Cone Geometry From the median values across all 193 frames:

$$\theta_c = [TBD] \text{ rad} = [TBD]^\circ \tag{4.2.1}$$

$$r = [TBD] \text{ mm} = [TBD] \text{ m} \tag{4.2.2}$$

$$A = \pi r^2 = [TBD] \text{ m}^2 \tag{4.2.3}$$

Rasmussen Pressure Coefficient Applying 2.1.9 with $M_\infty = 4.07$ and the measured cone half-angle:

$$C_p = \theta_c^2 \left[1 + \frac{(\gamma+1)M_\infty^2}{\theta_c^2 + 2} + \frac{(\gamma-1)M_\infty^2}{\theta_c^2+2} \ln! \left(\frac{\gamma+1}{2} + \frac{1}{M_\infty^2 \theta_c^2} \right) \right] = [TBD] \tag{4.2.4}$$

For slender conical flows, the surface pressure coefficient is uniform and equals the inviscid drag coefficient:

$$C_{D,\text{inviscid}} = C_p = [TBD] \tag{4.2.5}$$

Theoretical Drag Force and Acceleration Using the calculated freestream dynamic pressure $q_\infty = \frac{1}{2}\rho_\infty V_\infty^2$ and cone base area:

$$D_{\text{inviscid}} = C_{D,\text{inviscid}} \cdot q_\infty \cdot A = [TBD] \text{ N} \tag{4.2.6}$$

$$a_{\text{inviscid}} = \frac{D_{\text{inviscid}}}{m} = \frac{[TBD]}{0.0018\,\text{kg}} = [TBD] \text{ m/s}^2 \tag{4.2.7}$$

This theoretical prediction assumes inviscid flow and provides a baseline for comparison with the experimental drag measurement.

4.2.2 Experimental

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

4.3 Shock Angle

The theoretical shock angle was calculated using Rasmussen's approximation (2.1.8) for the measured cone half-angle and freestream Mach number. Applying this relationship:

$$\sin(\beta) = \sin(\theta)_c \sqrt{\frac{\gamma+1}{2} + \frac{1}{M_\infty^2 \sin^2\theta_c}} \tag{4.3.1}$$

With $M_\infty = 4.07$, $\theta_c = [TBD]$ rad, and $\gamma = 1.4$, the shock angle is:

$$\beta = [TBD] \text{ rad} = [TBD]^\circ \tag{4.3.2}$$

The shock wave forms a conical surface emanating from the cone vertex. In a two-dimensional side view (as captured in the shadowgraph images), this conical shock appears as two straight rays at angle β measured from the freestream direction. The theoretical shock geometry is visualized in 4.3.1, where the predicted shock rays (red dashed lines) are overlaid on the detected cone edges (blue solid lines) from a representative experimental frame.

The shock angle β is consistently larger than the cone surface angle θ_c, as required for attached shock conditions. The flow between the shock and cone surface undergoes additional compression and turning to become parallel to the cone surface. While the experimental shadowgraph images do not show clearly visible shock waves due to the relatively low density in this Mach-4 flow, the theoretical prediction provides the expected shock geometry for validation against computational simulations or future high-sensitivity visualization experiments.

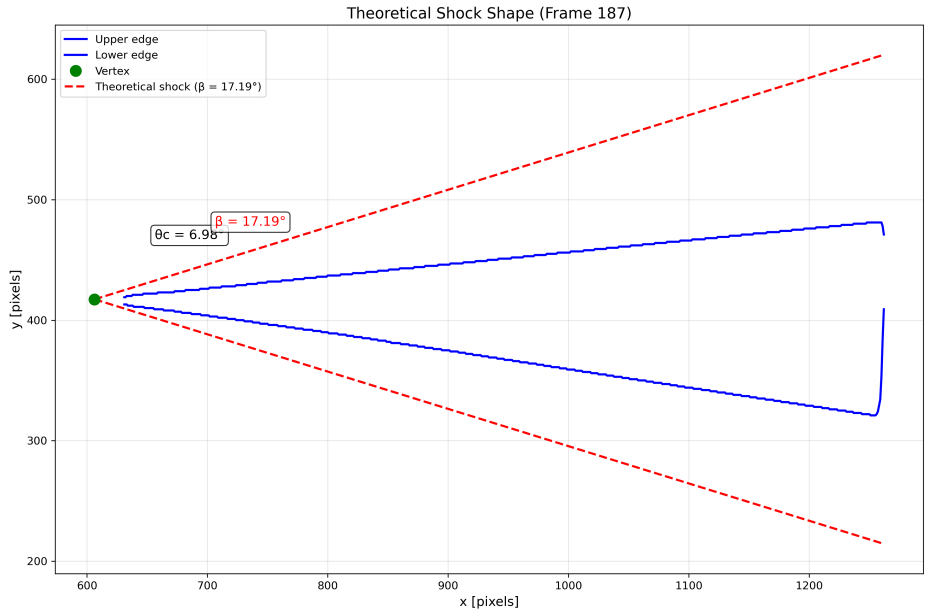

Figure 4.3.1: Theoretical shock shape overlaid on cone edges from experimental imaging. The blue solid lines show the detected cone edges, the green circle marks the calculated vertex position, and the red dashed lines represent the theoretical shock rays at angle $\beta = [TBD]^\circ$ from the freestream direction.

5 Analysis and Discussion

5.1 Comparison Between Theoretical and Experimental Drag Effects

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

5.2 Viscous Component of Drag

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

6 Summary and Conclusions

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

7 References

[1] S. J. Laurence and R. S. de Silva, *Ludwieg-tube experiments on a free-flying cone*, University of Maryland, April 8, 2026. [Online]. Available: https://github.com/vaisriv/enae464-lab05/blob/main/references/Ludwieg_tube_cone.pdf (see pp. 1, 2, 4).

[2] J. D. Anderson and C. P. Cadou, *Fundamentals of Aerodynamics*, 7th ed. McGraw-Hill Education, March 17, 2023, ISBN: 9781266076442 (see p. 1).

[3] S. J. Laurence, *Matlab cone*, April 13, 2026. [Online]. Available: https://github.com/vaisriv/enae464-lab05/blob/main/lib/matlab_cone.

8 Appendices

8.1 Data

For the sake of brevity, the raw measurement data will not be displayed in this report. However, in the spirit of completeness, the repository containing the complete data, source code, and notes for this report can be found at github:vaisriv/enae464-lab05.

8.2 Code

Similarly, only the code files that are key to the analysis are included below, and can be accessed in full in the aforementioned GitHub repository.

Listing 8.2.1: Data Analysis Script.
`./src/index.py`

```python
"""
ENAE464 Lab05: Ludwieg-Tube Experiments on a Free-Flying Cone
Analysis of a free-flying cone in a Mach-4 Ludwieg tube
"""

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker
from pathlib import Path
from scipy.optimize import fsolve
```

```python

# --- Paths -------------------------------------------------------------

DATA_DIR = Path("./data/mfg_m4_cone")
OUTPUT_TEXT_DIR = Path("./outputs/text")
OUTPUT_FIGS_DIR = Path("./outputs/figures")

OUTPUT_TEXT_DIR.mkdir(parents=True, exist_ok=True)
OUTPUT_FIGS_DIR.mkdir(parents=True, exist_ok=True)

# --- Header ------------------------------------------------------------

print("=" * 80)
print("ENAE464 Lab05: Ludwieg-Tube Experiments on a Free-Flying Cone")
print("=" * 80)
print()

# --- Constants ---------------------------------------------------------

# Physical constants
GAMMA = 1.4  # Specific heat ratio for air
R_GAS = 287.0  # Gas constant for air [J/(kg·K)]

# Experimental parameters (from experimental_setup.txt)
M_EXPERIMENTAL = 3.79  # Experimental test section Mach number
P_FILL = 0.7e5  # Fill pressure [Pa] (0.7 bar)
T_FILL = 296.0  # Fill temperature [K]
FRAME_RATE = 10000.0  # Camera frame rate [Hz]
CONE_MASS = 0.0018  # Cone mass [kg] (1.8 g)
PIXEL_TO_MM = 12.6  # Conversion factor [pixels/mm]
MM_PER_PIXEL = 1.0 / PIXEL_TO_MM  # Conversion factor [mm/pixel]
M_PER_PIXEL = MM_PER_PIXEL / 1000.0  # Conversion factor [m/pixel]

# Facility geometry
AREA_RATIO_TUBE = 11.4  # At/A* for charge tube

# Derived constants
DT = 1.0 / FRAME_RATE  # Time step between frames [s]

# ---  Functions ---------------------------------------------------------


def calculate_mt_from_area_ratio(area_ratio, gamma=GAMMA, tol=1e-8):
    """
    Solve for subsonic Mach number Mt in charge tube from area ratio.

    Uses Eq. 1: (A/A*)^2 = (1/M^2) * [2/(γ+1) * (1 + (γ-1)/2 * M^2)]^[(γ+1)/(γ-1)]

```

```python
    Parameters:
    -----------
    area_ratio : float
        At/A* ratio
    gamma : float
        Specific heat ratio
    tol : float
        Convergence tolerance for numerical solver

    Returns:
    --------
    float : Mt (subsonic solution)
    """
    from scipy.optimize import brentq

    def area_mach_relation(M, A_ratio, g):
        term1 = (2.0 / (g + 1.0)) * (1.0 + ((g - 1.0) / 2.0) * M**2)
        term2 = (g + 1.0) / (g - 1.0)
        return (1.0 / M**2) * term1**term2 - A_ratio**2

    # Use Brent's method with bounds to ensure positive subsonic solution
    # For subsonic flow, M is between a very small number and 1.0
    Mt = brentq(area_mach_relation, 0.001, 0.999, args=(area_ratio, gamma))

    return Mt


def calculate_expansion_conditions(Mt, p_fill, T_fill, gamma=GAMMA):
    """
    Calculate pressure and temperature behind expansion wave.

    Uses Eqs. 2-3:
    pt/p_fill = [1 + (γ-1)/2 * Mt^2]^[-2γ/(γ-1)]
    Tt/T_fill = [1 + (γ-1)/2 * Mt^2]^[-2]

    Parameters:
    -----------
    Mt : float
        Mach number in charge tube
    p_fill, T_fill : float
        Fill pressure [Pa] and temperature [K]
    gamma : float
        Specific heat ratio

    Returns:
    --------
    tuple : (pt, Tt)
    """
```

```python
    term = 1.0 + ((gamma - 1.0) / 2.0) * Mt**2

    pt = p_fill * term ** (-2.0 * gamma / (gamma - 1.0))
    Tt = T_fill * term ** (-2.0)

    return pt, Tt


def calculate_stagnation_conditions(Mt, pt, Tt, gamma=GAMMA):
    """
    Calculate stagnation pressure and temperature.

    Uses Eqs. 4-5:
    p0/pt = [1 + (γ-1)/2 * Mt^2]^[γ/(γ-1)]
    T0/Tt = 1 + (γ-1)/2 * Mt^2

    Parameters:
    -----------
    Mt : float
        Mach number in charge tube
    pt, Tt : float
        Pressure [Pa] and temperature [K] behind expansion
    gamma : float
        Specific heat ratio

    Returns:
    --------
    tuple : (p0, T0)
    """
    term = 1.0 + ((gamma - 1.0) / 2.0) * Mt**2

    p0 = pt * term ** (gamma / (gamma - 1.0))
    T0 = Tt * term

    return p0, T0


def calculate_freestream_conditions(M, p0, T0, R=R_GAS, gamma=GAMMA):
    """
    Calculate freestream static conditions from stagnation values.

    Uses Eqs. 7-8 plus ideal gas law and speed of sound.

    Parameters:
    -----------
    M : float
        Freestream Mach number
    p0, T0 : float
```

```python
        Stagnation pressure [Pa] and temperature [K]
    R : float
        Gas constant [J/(kg·K)]
    gamma : float
        Specific heat ratio

    Returns:
    --------
    dict : {'p': p_inf, 'T': T_inf, 'rho': rho_inf, 'V': V_inf, 'a': a_inf}
    """
    term = 1.0 + ((gamma - 1.0) / 2.0) * M**2

    p_inf = p0 / (term ** (gamma / (gamma - 1.0)))
    T_inf = T0 / term
    rho_inf = p_inf / (R * T_inf)
    a_inf = np.sqrt(gamma * R * T_inf)
    V_inf = M * a_inf

    return {"p": p_inf, "T": T_inf, "rho": rho_inf, "a": a_inf, "V": V_inf}


def load_edge_data(frame_num, edges_dir=DATA_DIR / "edges"):
    """Load upper and lower edge coordinates for a single frame."""
    upper_file = edges_dir / f"frame{frame_num}_upper.txt"
    lower_file = edges_dir / f"frame{frame_num}_lower.txt"

    upper = pd.read_csv(upper_file, sep="\t")
    lower = pd.read_csv(lower_file, sep="\t")

    return upper, lower


def fit_cone_edges_and_find_vertex(upper, lower):
    """
    Fit straight lines to upper and lower edges, find vertex.

    Returns:
    --------
    dict with keys:
        'vertex_x', 'vertex_y': vertex coordinates [pixels]
        'upper_slope', 'lower_slope': edge slopes
        'upper_intercept', 'lower_intercept': edge y-intercepts
        'cone_angle_rad', 'cone_angle_deg': half-angle
        'base_height_pixels': vertical distance between edges at back
    """
    # Fit lines: y = slope * x + intercept
    upper_coeffs = np.polyfit(upper["x"], upper["y"], 1)
    lower_coeffs = np.polyfit(lower["x"], lower["y"], 1)
```

```python

    m_upper, b_upper = upper_coeffs
    m_lower, b_lower = lower_coeffs

    # Find intersection (vertex)
    # m_upper * x + b_upper = m_lower * x + b_lower
    vertex_x = (b_lower - b_upper) / (m_upper - m_lower)
    vertex_y = m_upper * vertex_x + b_upper

    # Cone half-angle from average slope
    avg_slope = (abs(m_upper) + abs(m_lower)) / 2.0
    cone_angle_rad = np.arctan(avg_slope)
    cone_angle_deg = np.degrees(cone_angle_rad)

    # Base height (at rightmost x position) - use actual edge points, not extrapolation
    x_max = min(upper["x"].max(), lower["x"].max())

    # Find actual edge points near the base (within 5 pixels of x_max)
    upper_base_points = upper[upper["x"] ≥ x_max - 5]
    lower_base_points = lower[lower["x"] ≥ x_max - 5]

    # Get actual y values at the base
    if len(upper_base_points) > 0 and len(lower_base_points) > 0:
        y_upper_base = upper_base_points["y"].mean()
        y_lower_base = lower_base_points["y"].mean()
        base_height_pixels = abs(y_upper_base - y_lower_base)
    else:
        # Fallback to extrapolation if no points found
        y_upper_base = m_upper * x_max + b_upper
        y_lower_base = m_lower * x_max + b_lower
        base_height_pixels = abs(y_upper_base - y_lower_base)

    return {
        "vertex_x": vertex_x,
        "vertex_y": vertex_y,
        "upper_slope": m_upper,
        "lower_slope": m_lower,
        "upper_intercept": b_upper,
        "lower_intercept": b_lower,
        "cone_angle_rad": cone_angle_rad,
        "cone_angle_deg": cone_angle_deg,
        "base_height_pixels": base_height_pixels,
    }


def rasmussen_pressure_coefficient(M, theta_c, gamma=GAMMA):
    """
    Calculate pressure coefficient using Rasmussen's approximation (Eq. 11).
```

```python

    Cp/θc² ≈ 1 + [(γ+1)M²]/(θc²+2) + [(γ-1)M²]/(θc²+2)*ln[(γ+1)/2 + 1/(M²θc²)]

    Parameters:
    -----------
    M : float
        Freestream Mach number
    theta_c : float
        Cone half-angle [radians]
    gamma : float
        Specific heat ratio

    Returns:
    --------
    float : Cp (pressure coefficient)
    """
    theta_c_sq = theta_c**2

    term1 = 1.0
    term2 = ((gamma + 1.0) * M**2) / (theta_c_sq + 2.0)

    log_arg = (gamma + 1.0) / 2.0 + 1.0 / (M**2 * theta_c_sq)
    term3 = ((gamma - 1.0) * M**2) / (theta_c_sq + 2.0) * np.log(log_arg)

    Cp_over_theta_sq = term1 + term2 + term3
    Cp = Cp_over_theta_sq * theta_c_sq

    return Cp


def rasmussen_shock_angle(M, theta_c, gamma=GAMMA):
    """
    Calculate shock angle using Rasmussen's approximation (Eq. 10).

    sin(β)/sin(θc) ≈ √[(γ+1)/2 + 1/(M² sin²θc)]

    Parameters:
    -----------
    M : float
        Freestream Mach number
    theta_c : float
        Cone half-angle [radians]
    gamma : float
        Specific heat ratio

    Returns:
    --------
    float : β (shock angle) [radians]
```

```python
    """
    sin_theta_c = np.sin(theta_c)

    term = (gamma + 1.0) / 2.0 + 1.0 / (M**2 * sin_theta_c**2)
    sin_beta = sin_theta_c * np.sqrt(term)

    beta = np.arcsin(sin_beta)

    return beta


# --- Analysis ---------------------------------------------------------------

print("1. Calculating Freestream Conditions")
print("-" * 80)

# Step 1: Tube Mach number
Mt = calculate_mt_from_area_ratio(AREA_RATIO_TUBE, GAMMA)
print(f"  Mt (charge tube Mach number): {Mt:.6f}")

# Step 2: Expansion wave conditions
pt, Tt = calculate_expansion_conditions(Mt, P_FILL, T_FILL, GAMMA)
print(f"  pt (pressure behind expansion): {pt:.2f} Pa")
print(f"  Tt (temperature behind expansion): {Tt:.2f} K")

# Step 3: Stagnation conditions
p0, T0 = calculate_stagnation_conditions(Mt, pt, Tt, GAMMA)
print(f"  p0 (stagnation pressure): {p0:.2f} Pa")
print(f"  T0 (stagnation temperature): {T0:.2f} K")

# Step 4: Freestream conditions
freestream = calculate_freestream_conditions(M_EXPERIMENTAL, p0, T0, R_GAS, GAMMA)
print(f"  M∞ (freestream Mach number): {M_EXPERIMENTAL}")
print(f"  p∞ (freestream pressure): {freestream['p']:.2f} Pa")
print(f"  T∞ (freestream temperature): {freestream['T']:.2f} K")
print(f"  ρ∞ (freestream density): {freestream['rho']:.6f} kg/m³")
print(f"  a∞ (freestream sound speed): {freestream['a']:.2f} m/s")
print(f"  V∞ (freestream velocity): {freestream['V']:.2f} m/s")
print()

print("2. Loading and Processing Edge Detection Data")
print("-" * 80)

# Determine available frames
edges_dir = DATA_DIR / "edges"
frame_files = sorted(edges_dir.glob("frame*_upper.txt"))
frame_numbers = [
    int(f.stem.split("_")[0].replace("frame", "")) for f in frame_files
```

```python
]
n_frames = len(frame_numbers)

print(f"  Found {n_frames} frames with edge data")
print(f"  Frame range: {min(frame_numbers)} to {max(frame_numbers)}")
print()

print("3. Extracting Cone Geometry")
print("-" * 80)

# Extract geometry from all frames
geometries = []
for frame_num in frame_numbers:
    upper, lower = load_edge_data(frame_num)
    geom = fit_cone_edges_and_find_vertex(upper, lower)
    geom["frame"] = frame_num
    geom["time"] = frame_num * DT
    geometries.append(geom)

geom_df = pd.DataFrame(geometries)

# Calculate median cone angle from all frames
median_cone_angle_deg = geom_df["cone_angle_deg"].median()
median_cone_angle_rad = geom_df["cone_angle_rad"].median()

# Use base height from FIRST 20 frames (cone is closest to camera, least perspective error)
initial_base_height_px = geom_df.iloc[:20]["base_height_pixels"].median()
base_radius_m = (initial_base_height_px / 2.0) * M_PER_PIXEL

print(
    f"  Median cone half-angle: {median_cone_angle_deg:.3f}° ({median_cone_angle_rad:.6f} rad)"
)
print(f"  Initial base diameter (first 20 frames): {initial_base_height_px:.2f} pixels")
print(f"  Base radius: {base_radius_m*1000:.3f} mm ({base_radius_m:.6f} m)")
print(f"  Base area: {np.pi * base_radius_m**2:.6e} m²")
print()

# Store for later use
THETA_C = median_cone_angle_rad
R_BASE = base_radius_m
A_BASE = np.pi * R_BASE**2

print("4. Analyzing Cone Motion and Calculating Drag")
print("-" * 80)

# Extract vertex positions over time (convert to meters)
time_array = geom_df["time"].values
```

```python
x_position_pixels = geom_df["vertex_x"].values
x_position_m = x_position_pixels * M_PER_PIXEL

# Fit quadratic polynomial: x = a0 + a1*t + a2*t^2
coeffs = np.polyfit(time_array, x_position_m, 2)
a2, a1, a0 = coeffs  # Note: polyfit returns highest degree first

print(f"  Quadratic fit coefficients:")
print(f"    a0 = {a0:.6f} m")
print(f"    a1 = {a1:.6f} m/s")
print(f"    a2 = {a2:.6f} m/s²")
print()

# Calculate acceleration
ax_experimental = 2.0 * a2

print(f"  Experimental acceleration: ax = {ax_experimental:.3f} m/s²")
print()

# Calculate drag force
drag_force_experimental = CONE_MASS * ax_experimental

print(f"  Experimental drag force: D = {drag_force_experimental:.6f} N")
print()

# Calculate experimental drag coefficient
q_inf = 0.5 * freestream["rho"] * freestream["V"] ** 2  # Dynamic pressure
CD_experimental = drag_force_experimental / (q_inf * A_BASE)

print(f"  Dynamic pressure: q∞ = {q_inf:.2f} Pa")
print(f"  Experimental drag coefficient: CD = {CD_experimental:.6f}")
print()

# Generate fit curve for visualization
x_fit = a0 + a1 * time_array + a2 * time_array**2

print("5. Calculating Theoretical Drag (Rasmussen Approximation)")
print("-" * 80)

# Calculate Cp
Cp_theory = rasmussen_pressure_coefficient(M_EXPERIMENTAL, THETA_C, GAMMA)
print(f"  Rasmussen Cp (inviscid): {Cp_theory:.6f}")
print()

# For slender cones, Cp ≈ CD_inviscid
CD_inviscid = Cp_theory

# Calculate theoretical drag force
```

```python
drag_force_inviscid = CD_inviscid * q_inf * A_BASE

print(f"  Inviscid drag coefficient: CD_inviscid = {CD_inviscid:.6f}")
print(f"  Inviscid drag force: D_inviscid = {drag_force_inviscid:.6f} N")
print()

# Calculate theoretical acceleration
ax_inviscid = drag_force_inviscid / CONE_MASS

print(f"  Inviscid acceleration: ax_inviscid = {ax_inviscid:.3f} m/s²")
print()

print("6. Estimating Viscous Drag Component")
print("-" * 80)

# Viscous drag is the difference between experimental and inviscid
drag_force_viscous = drag_force_experimental - drag_force_inviscid
CD_viscous = CD_experimental - CD_inviscid

# Percentage breakdown
if drag_force_experimental ≠ 0:
    viscous_fraction = drag_force_viscous / drag_force_experimental * 100.0
    inviscid_fraction = drag_force_inviscid / drag_force_experimental * 100.0
else:
    viscous_fraction = 0.0
    inviscid_fraction = 0.0

print(f"  Total drag (experimental): {drag_force_experimental:.6f} N")
print(
    f"  Inviscid drag (theoretical): {drag_force_inviscid:.6f} N ({inviscid_fraction:.1f}%)"
)
print(
    f"  Viscous drag (estimated): {drag_force_viscous:.6f} N ({viscous_fraction:.1f}%)"
)
print()
print(f"  CD_experimental: {CD_experimental:.6f}")
print(f"  CD_inviscid: {CD_inviscid:.6f}")
print(f"  CD_viscous: {CD_viscous:.6f}")
print()

print("7. Calculating Theoretical Shock Angle")
print("-" * 80)

# Calculate shock angle
beta_rad = rasmussen_shock_angle(M_EXPERIMENTAL, THETA_C, GAMMA)
beta_deg = np.degrees(beta_rad)

print(f"  Cone half-angle: θc = {np.degrees(THETA_C):.3f}°")
```

```python
print(f"  Shock angle: β = {beta_deg:.3f}° ({beta_rad:.6f} rad)")
print()

# --- Visualization ------------------------------------------------------------

print("8. Generating Figures")
print("-" * 80)

# Figure 1: Position vs Time
fig, ax = plt.subplots(figsize=(10, 6))
ax.plot(
    time_array * 1000,
    x_position_m * 1000,
    "o",
    label="Measured vertex position",
    markersize=4,
    alpha=0.6,
)
ax.plot(
    time_array * 1000,
    x_fit * 1000,
    "-",
    label=f"Quadratic fit: $x = {a0:.3f} + {a1:.3f}t + {a2:.3f}t^2$",
    linewidth=2,
    color="red",
)
ax.set_xlabel("Time [ms]", fontsize=12)
ax.set_ylabel("Position [mm]", fontsize=12)
ax.set_title("Cone Vertex Position vs. Time", fontsize=14)
ax.legend(fontsize=10)
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig(OUTPUT_FIGS_DIR / "position_vs_time.png", dpi=300)
plt.close()
print("  Saved: position_vs_time.png")

# Figure 2: Velocity vs Time
velocity_measured = np.gradient(x_position_m, time_array)
velocity_fit = a1 + 2 * a2 * time_array

fig, ax = plt.subplots(figsize=(10, 6))
ax.plot(
    time_array * 1000,
    velocity_measured,
    "o",
    label="Numerical derivative",
    markersize=3,
    alpha=0.5,
```

```python
)
ax.plot(
    time_array * 1000,
    velocity_fit,
    "-",
    label=f"From fit: $v = {a1:.3f} + 2({a2:.3f})t$",
    linewidth=2,
    color="red",
)
ax.set_xlabel("Time [ms]", fontsize=12)
ax.set_ylabel("Velocity [m/s]", fontsize=12)
ax.set_title("Cone Velocity vs. Time", fontsize=14)
ax.legend(fontsize=10)
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig(OUTPUT_FIGS_DIR / "velocity_vs_time.png", dpi=300)
plt.close()
print("  Saved: velocity_vs_time.png")

# Figure 3: Acceleration vs Time
acceleration_measured = np.gradient(velocity_measured, time_array)

fig, ax = plt.subplots(figsize=(10, 6))
ax.plot(
    time_array * 1000,
    acceleration_measured,
    "o",
    label="From numerical derivatives",
    markersize=3,
    alpha=0.5,
)
ax.axhline(
    y=ax_experimental,
    color="red",
    linestyle="--",
    linewidth=2,
    label=f"From quadratic fit: $a_x = {ax_experimental:.3f}$ m/s²",
)
ax.set_xlabel("Time [ms]", fontsize=12)
ax.set_ylabel("Acceleration [m/s²]", fontsize=12)
ax.set_title("Cone Acceleration vs. Time", fontsize=14)
ax.legend(fontsize=10)
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig(OUTPUT_FIGS_DIR / "acceleration_vs_time.png", dpi=300)
plt.close()
print("  Saved: acceleration_vs_time.png")
```

```python
# Figure 4: Theoretical shock shape overlay
# Select a frame near the middle with low angle of attack
middle_frame = frame_numbers[len(frame_numbers) // 2]
upper_mid, lower_mid = load_edge_data(middle_frame)
geom_mid = geom_df[geom_df["frame"] == middle_frame].iloc[0]

fig, ax = plt.subplots(figsize=(12, 8))

# Plot cone edges
ax.plot(upper_mid["x"], upper_mid["y"], "b-", linewidth=2, label="Upper edge")
ax.plot(lower_mid["x"], lower_mid["y"], "b-", linewidth=2, label="Lower edge")

# Plot vertex
vx, vy = geom_mid["vertex_x"], geom_mid["vertex_y"]
ax.plot(vx, vy, "go", markersize=10, label="Vertex", zorder=5)

# Plot theoretical shock rays
x_shock_range = np.linspace(vx, upper_mid["x"].max(), 200)

# Upper shock: y = vy + (x - vx) * tan(β)
y_shock_upper = vy + (x_shock_range - vx) * np.tan(beta_rad)

# Lower shock: y = vy - (x - vx) * tan(β)
y_shock_lower = vy - (x_shock_range - vx) * np.tan(beta_rad)

ax.plot(
    x_shock_range,
    y_shock_upper,
    "r--",
    linewidth=2,
    label=f"Theoretical shock (β = {beta_deg:.2f}°)",
)
ax.plot(x_shock_range, y_shock_lower, "r--", linewidth=2)

# Annotations
ax.text(
    vx + 50,
    vy + 50,
    f"θc = {np.degrees(THETA_C):.2f}°",
    fontsize=12,
    bbox=dict(boxstyle="round", facecolor="white", alpha=0.8),
)
ax.text(
    vx + 100,
    vy + (x_shock_range[50] - vx) * np.tan(beta_rad) + 10,
    f"β = {beta_deg:.2f}°",
    fontsize=12,
    color="red",
```

```
    bbox=dict(boxstyle="round", facecolor="white", alpha=0.8),
)

ax.set_xlabel("x [pixels]", fontsize=12)
ax.set_ylabel("y [pixels]", fontsize=12)
ax.set_title(f"Theoretical Shock Shape (Frame {middle_frame})", fontsize=14)
ax.legend(fontsize=10)
ax.axis("equal")
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig(OUTPUT_FIGS_DIR / "theoretical-shock.png", dpi=300)
plt.close()
print("  Saved: theoretical-shock.png (updated)")

# Figure 5: Drag coefficient comparison bar chart
fig, ax = plt.subplots(figsize=(8, 6))

categories = ["Experimental\n(Total)", "Inviscid\n(Theory)", "Viscous\n(Estimated)"]
values = [CD_experimental, CD_inviscid, CD_viscous]
colors = ["#1f77b4", "#ff7f0e", "#2ca02c"]

bars = ax.bar(categories, values, color=colors, alpha=0.7, edgecolor="black")

# Add value labels on bars
for bar, val in zip(bars, values):
    height = bar.get_height()
    ax.text(
        bar.get_x() + bar.get_width() / 2.0,
        height,
        f"{val:.4f}",
        ha="center",
        va="bottom",
        fontsize=11,
        fontweight="bold",
    )

ax.set_ylabel("Drag Coefficient", fontsize=12)
ax.set_title("Drag Coefficient Comparison", fontsize=14)
ax.grid(True, axis="y", alpha=0.3)
plt.tight_layout()
plt.savefig(OUTPUT_FIGS_DIR / "drag_coefficient_comparison.png", dpi=300)
plt.close()
print("  Saved: drag_coefficient_comparison.png")

# Figure 6: Cone angle variation (quality check)
fig, ax = plt.subplots(figsize=(10, 6))
ax.plot(geom_df["time"] * 1000, geom_df["cone_angle_deg"], "o-", markersize=3)
ax.axhline(
```

```python
    y=median_cone_angle_deg,
    color="red",
    linestyle="--",
    linewidth=2,
    label=f"Median: {median_cone_angle_deg:.3f}°",
)
ax.set_xlabel("Time [ms]", fontsize=12)
ax.set_ylabel("Measured Cone Half-Angle [°]", fontsize=12)
ax.set_title("Cone Half-Angle Over Time (Measurement Consistency)", fontsize=14)
ax.legend(fontsize=10)
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig(OUTPUT_FIGS_DIR / "cone_angle_consistency.png", dpi=300)
plt.close()
print("  Saved: cone_angle_consistency.png")

print()

# --- Output ------------------------------------------------------------------

print("9. Exporting Data to CSV")
print("-" * 80)

# Table 1: Freestream conditions
freestream_data = {
    "Parameter": [
        "M_t",
        "p_t [Pa]",
        "T_t [K]",
        "p_0 [Pa]",
        "T_0 [K]",
        "M_∞ [experimental]",
        "p_∞ [Pa]",
        "T_∞ [K]",
        "ρ_∞ [kg/m³]",
        "a_∞ [m/s]",
        "V_∞ [m/s]",
    ],
    "Value": [
        Mt,
        pt,
        Tt,
        p0,
        T0,
        M_EXPERIMENTAL,
        freestream["p"],
        freestream["T"],
        freestream["rho"],
```

```
        freestream["a"],
        freestream["V"],
    ],
}
df_freestream = pd.DataFrame(freestream_data)
df_freestream.to_csv(OUTPUT_TEXT_DIR / "freestream_conditions.csv", index=False)
print("  Saved: freestream_conditions.csv")

# Table 2: Cone geometry
cone_geom_data = {
    "Parameter": [
        "Half-angle [rad]",
        "Half-angle [°]",
        "Base radius [m]",
        "Base radius [mm]",
        "Base area [m²]",
    ],
    "Value": [
        THETA_C,
        np.degrees(THETA_C),
        R_BASE,
        R_BASE * 1000,
        A_BASE,
    ],
}
df_cone_geom = pd.DataFrame(cone_geom_data)
df_cone_geom.to_csv(OUTPUT_TEXT_DIR / "cone_geometry.csv", index=False)
print("  Saved: cone_geometry.csv")

# Table 3: Drag analysis summary
drag_summary_data = {
    "Parameter": [
        "Quadratic fit a0 [m]",
        "Quadratic fit a1 [m/s]",
        "Quadratic fit a2 [m/s²]",
        "Experimental acceleration [m/s²]",
        "Experimental drag force [N]",
        "Experimental drag coefficient",
        "Rasmussen Cp (inviscid)",
        "Inviscid drag force [N]",
        "Inviscid drag coefficient",
        "Viscous drag force [N]",
        "Viscous drag coefficient",
        "Viscous fraction [%]",
    ],
    "Value": [
        a0,
        a1,
```

```
        a2,
        ax_experimental,
        drag_force_experimental,
        CD_experimental,
        Cp_theory,
        drag_force_inviscid,
        CD_inviscid,
        drag_force_viscous,
        CD_viscous,
        viscous_fraction,
    ],
}
df_drag = pd.DataFrame(drag_summary_data)
df_drag.to_csv(OUTPUT_TEXT_DIR / "drag_analysis.csv", index=False)
print("  Saved: drag_analysis.csv")

# Table 4: Shock angle
shock_data = {
    "Parameter": [
        "Cone half-angle [rad]",
        "Cone half-angle [°]",
        "Shock angle [rad]",
        "Shock angle [°]",
    ],
    "Value": [THETA_C, np.degrees(THETA_C), beta_rad, beta_deg],
}
df_shock = pd.DataFrame(shock_data)
df_shock.to_csv(OUTPUT_TEXT_DIR / "shock_angle.csv", index=False)
print("  Saved: shock_angle.csv")

# Table 5: Full trajectory data
geom_df.to_csv(OUTPUT_TEXT_DIR / "trajectory_data.csv", index=False)
print("  Saved: trajectory_data.csv")

print()
print("=" * 80)
print("Analysis Complete!")
print("=" * 80)
```